

# Tech Spec : Migrating existing POS merchants to Batch clearing service.

Author/s: Manoj Kumar V  
Team/Pod: Razorpay POS | BU: Payments  
Published Date:

| Reviewer Name  | Reviewed Date | Status        |
|----------------|---------------|---------------|
| Sunny Aggrawal |               | Not started ▾ |
| Karthik S      |               | Not started ▾ |

## 1. Problem Statement

Currently, migrating merchants from the Pos stack to Single stack has two areas for improvement. First, merchant migrations are dependent on end-of-day batch clearing processes, limiting our flexibility and causing delays. Second, performing batch clearing through the Ezetap system, even post-migration, keeps systems tied to legacy complexities and processes like terminal locking and key exchanges have to be solved for newer systems. By migrating batch clearing into batch-clearing service before merchant migration, we can migrate merchants at any convenient time, removing the dependency on end-of-day processes. Additionally, systems will benefit from Razorpay's infrastructure without relying on legacy components.

## 2. Introduction and Scope

In most payment gateways, batch clearing happens at the terminal level. Since we move the same terminals along with the merchant, it's crucial to make sure all previous transactions on those terminals are settled before the migration.

If we skip batch clearing, the older transactions might not be available in the new system, and that can cause issues with settlement. To avoid this, we currently wait for batch clearing to complete in the existing system before migrating the merchant. This ensures that everything is cleanly wrapped up before the move, and the merchant can continue processing transactions smoothly on the new stack.

**Scope:**

1. Merchant Level migration.
2. PG Level migration
3. Auto settlements will happen on batch-clearing service
4. Dual stack merchants also to be moved to batch-clearing service
5. The batch clearing service will handle scheduling (crons) and configuration management.

**Out of Scope:**

1. Post processings
2. Manual Settlements
3. Org Level Settlements

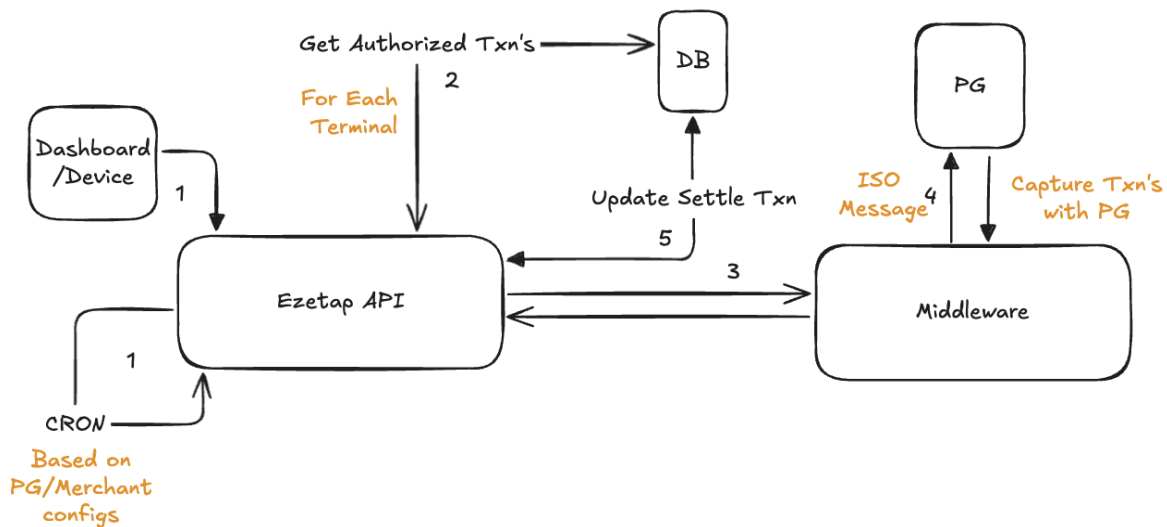
### 3. Assumptions

1. The migration of merchants is from pos to single stack and not from pos to dual stack.
2. Merchant migration SOP interface will inform batch-clearing service incase of any merchant migration.
3. If merchants are migrated without batch clearing the existing transactions, then migrating existing terminal keys also has to happen \*\*.
- 4.

## 4. Current Architecture

### 4.1 Pos Stack

#### 4.1.1 HLD



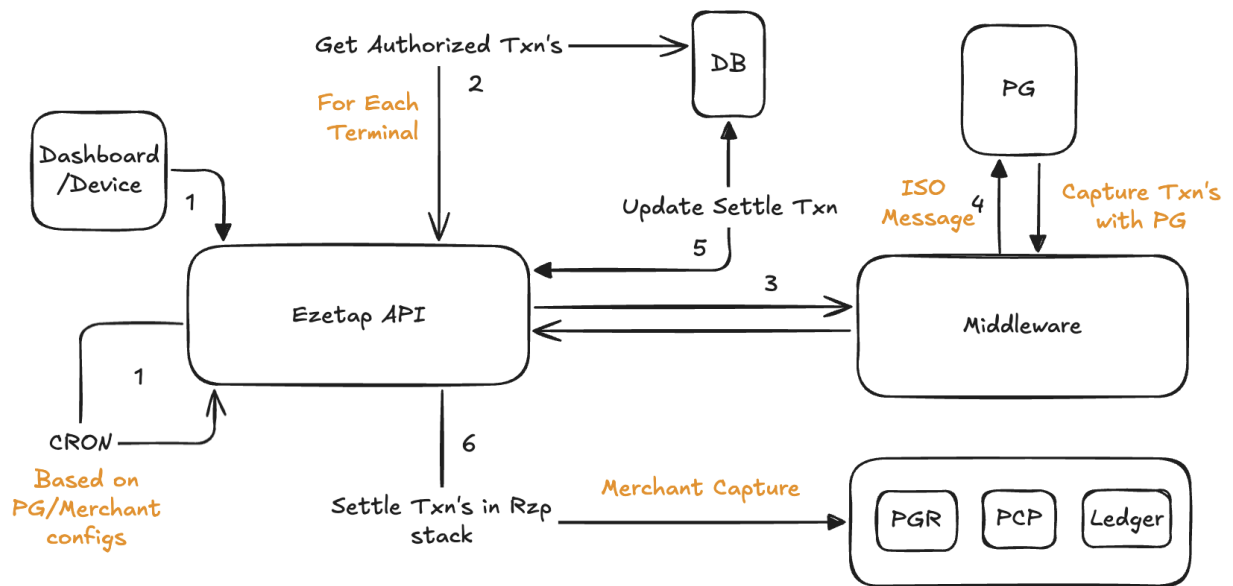
#### Ezetap API Responsibilities:

1. Picks all terminals with not settled flag. Filters out terminals which are part of orgLevelSettlement.
2. Terminal Locking: It acquires terminal lock but a different logic is written based on TERMINAL\_RESTRICTION settings. - Add section for this
3. Filter refund transactions - Config stored at PG level
4. Handle pending transactions - Calls the reversal API for pending transactions and removed preAuth transactions
5. Handle zero transactions - Even if there are no transactions to settle, we still update terminal as settled and mark device as settled, enables key exchange as true and expire device login
6. Calls MWare to Settle transactions
7. If middleware response is Success, it updates terminal info - it marks it as settled and enables key exchange as true
8. Updates transaction states in Ezetap to Settled. Reset fields, login in device.
9. Unlock terminal
10. Raise reposting - For omni merchants to capture payments on Rzp side.

Complete Doc on Existing Flow: [LINK](#)

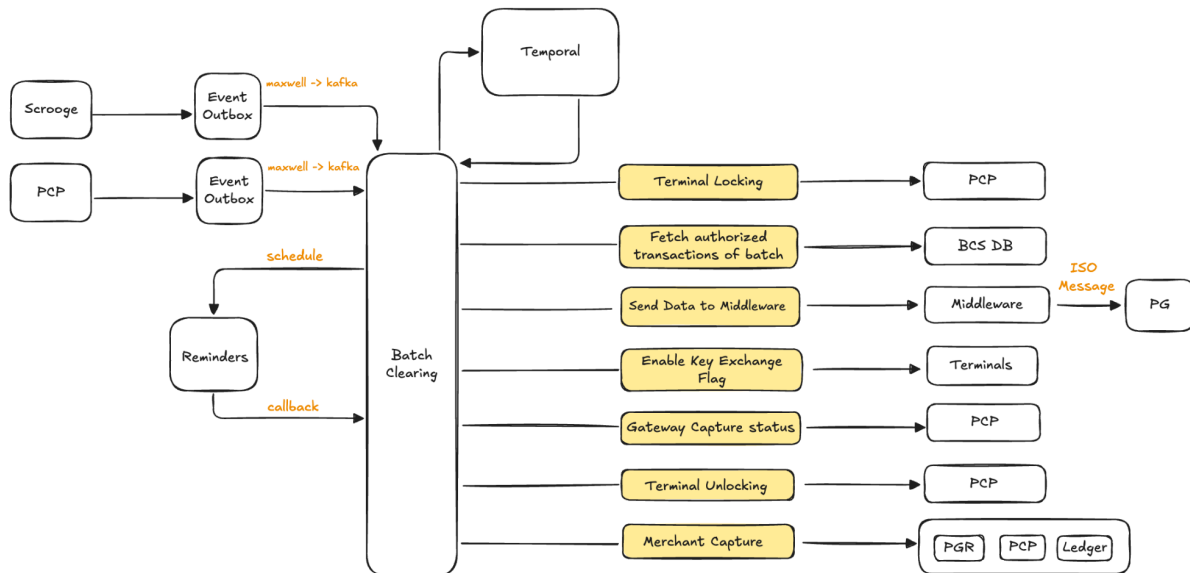
## 4.2 Omni Stack

### 4.2.1 HLD

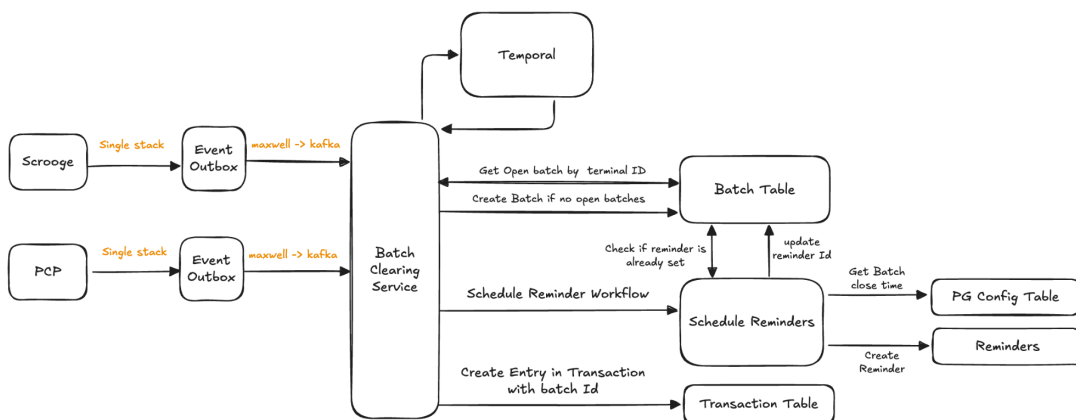


## 4.3 Single Stack

### 4.3.1 HLD



### 4.3.2 Batch creation flow



## 5. Final Approach

### 5.1 Data flow from Ezetap to Batch clearing service

Batch clearing happens at the terminal level, meaning all unsettled transactions linked to a terminal must be captured at one place. If batch-clearing is to be migrated to the batch-clearing service, it must have access to all unsettled transactions at the terminal level. This data can either be pulled from the Ezetap API or pushed from Ezetap to the batch-clearing service.

#### 5.1.1 Pull based approach [Not Preferred]

In this approach, whenever batch-clearing is triggered at the batch-clearing service, it must pull unsettled transaction data from the Ezetap API and process batch clearing.

1. Pulling Data on PG Level: We need to pull the data from Ezetap API and store them in batch clearing at PG Level. Currently for Amex batch-clearing, we are fetching the data on payment-gateway and acquirer level. Amex has very limited transactions (~2k per day) and for other PG's like HDFC, this number is huge (~25k per day). So pulling this data at once is not preferred.
2. Pulling data on TID and MID Level: We can pull the data on TID and MID wise from Ezetap API. But batch-clearing should have this TID and MID data beforehand which is not anywhere in the scope of current batch-clearing service architecture. Even if we prefill this TID and MID data, any new terminals onboarded on Ezetap side should also be added to batch-clearing service which again becomes a tedious job.

#### 5.1.2 Push based approach

An alternative is a push-based mechanism, where Ezetap actively sends transaction data to the batch-clearing service through an event-driven architecture.

1. Ezetap API can send data to batch clearing service via async queues. Each time a transaction is completed on Ezetap, the relevant data is pushed to a queue with all required data.
2. Batch clearing service then consumes the data and stores it in its tables.
3. Whenever batch clearing happens, the service simply fetches the pre-stored transaction data and performs batch clearing.

Using AWS SQS for Push based Approach

1. 1SQS is already integrated with Ezetap Infrastructure and it should be simple to create a new producer and use it.

2. The batch-clearing service will act as a consumer, processing and storing transactions as they arrive.
3. Since Ezetap is a deprecating stack, investing in a full-fledged event streaming system (e.g., Maxwell + Kafka) would be high effort with low long-term benefits.

#### Cross Account Access for SQS

1. Since Ezetap and Batch-clearing service use different AWS accounts, we need to set up Cross account access for SQS.
2. Batch-clearing should own this SQS and Ezetap API IAM role should be added for write access.

### Using Kafka for Push based Approach [Preferred]

1. Kafka is already integrated with Batch clearing service and it should be simple to create a new consumer and use it.
2. The Ezetap service will act as a producer
3. We can use the same existing topic (batch-clearing-events) ?? This will help us in not creating new consumer. But some business logic has to be changed to consume Ezetap transactions

#### Cross Account Access for Kafka

1. Since Ezetap and Batch-clearing service use different AWS accounts, we need to set up Cross account access for Kafka.

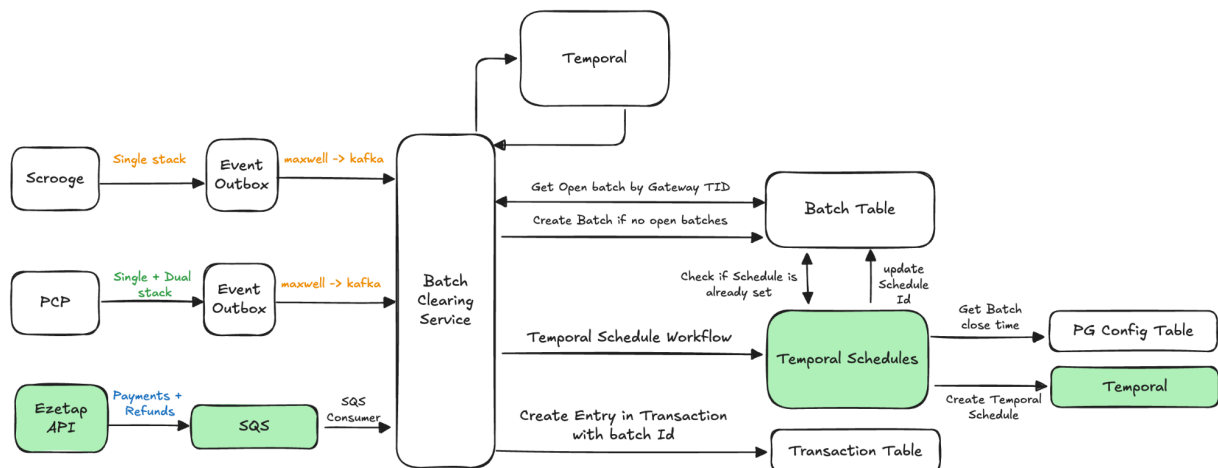
### Sample Payload

```
{
  "tid":1234,
  "mid":"M1233",
  "payment_gateway":"HDFC",
  "acquirer":"HDFC",
  "type":"CHARGE",
  "txn_id":"250325113000665E411407793",
  "amount":"100",
  "status":"AUTHORISED"
}
```

## 5.2 Batch creation changes in BCS

Currently, batch creation is tied to a terminalId. Whenever a terminal does not have an open batch, a new batch is created, and reminders are scheduled accordingly. However, for POS stack transactions, terminalId will not be available because terminals are not created on the Razorpay side.

1. Move batch creation logic to Gateway TID + MID Level
  - a. Instead of relying on terminalId, batch creation should be based on Gateway TID (Terminal ID) and MID (Merchant ID), as this combination is always unique.
  - b. This ensures that batch clearing can happen regardless of whether a terminal exists on Razorpay's side.
2. Handling Optional Fields for POS Stack Transactions
  - a. Since POS stack transactions do not have terminalId and merchantId, these should be treated as optional fields during batch creation.
  - b. The batch-clearing service should be able to process batches without these fields and rely on TID and MID for uniquely identifying transactions.



## 5.3 Event consumption in BCS

The batch clearing service should now consume Pos stack transactions. A new Queue consumer should be created to consume these transactions. Whenever a new transaction is pushed from the Ezetap API side, batch clearing should consume them and create batches accordingly.

1. Creating a New Queue Consumer for POS Transactions
  - a. A dedicated Queue consumer should be implemented to ingest POS transactions from Ezetap.



- b. These transactions will be processed and grouped into batches by the batch clearing service.
- 2. Storing Transaction Source in Metadata
  - a. Each transaction's origin (POS or Single Stack) should be recorded in the metadata field of the transactions table.
  - b. This will allow the system to determine where updates should be applied in respective stacks.
- 3. Storing Merchant context in Batch
  - a. While creating a batch for new transactions, we need to store merchant context (which stack the merchant is in). This will be stored in the meta\_data column of the batch table. This is required to perform terminal locking and enabling key exchange in respective stacks
- 4. Handling Batch Updates for Migrated Merchants
  - a. In the current event consumer for single stack transactions, once the batch is created, it won't be updated until closed except for reminders.
  - b. This needs a change here because say if the batch is created by pos transactions and the merchant has been moved to single stack and he started doing transactions. Now the batch entity won't be having all the required fields (terminalId, merchantId) for single stack batch-clearing.
  - c. To fix this, when consuming Single Stack transactions, the batch clearing service should detect and update missing fields (e.g., terminalId, merchantId) in existing batch records.

## 5.4 Merchant context in BCS

Batch clearing service should have the context of whether the merchant is on single stack or pos stack because terminal locking and enabling key exchange should happen on respective stacks.

### How to Determine Merchant Context?

The batch clearing service needs a reliable way to identify a merchant's current stack. There are two possible approaches:

1. Event-Based Approach: BCS could receive an event notification whenever a merchant is moved across stacks (from POS to Single Stack or vice versa). This ensures real-time updates but requires additional event tracking infrastructure.

### New API in Batch-clearing to record Merchant context

- a. Merchant Migration Notification: Whenever a merchant migrates, the Batch-Clearing Service should be notified by calling this API. The API should

accept TID (Terminal ID), MID (Merchant ID), RzpTerminalId, RzpMerchantId, and the target stack the merchant is moving to.

- b. Recording Merchant Migration: The Batch-Clearing Service should record the merchant's migration against an open batch. The metadata column in the batch table should store this merchant context, specifying which stack the batch-clearing process should occur in.
  - c. Handling Transactions Without Open Batches: If there are no open batches for the given MID + TID, the record will be ignored. When a new transaction occurs, the merchant context will be recorded as part of the event consumption flow.
  - d. Utilizing Merchant Context for Processing: The merchant context will be used to determine where Terminal Locking and Enable Key Exchange should take place. It will also help in storing RzpTerminalId and RzpMerchantId in cases where a merchant moves to a single stack.
2. Service Call Approach: BCS can call an external service or API to check if a merchant is in Single Stack or POS Stack before performing batch clearing.

### **Existing Flags for Merchant Context**

Currently, both Ezetap and Razorpay ecosystems store merchant context through flags. Ezetap maintains Single Stack Enabled settings, which indicate whether a merchant is in Single Stack or POS Stack. Razorpay uses a DCS flag to determine if a merchant is in Single Stack.

1. Ezetap Settings (POS Stack Side) **[Preferred]**
  - a. On Ezetap's side, merchants have a "Single Stack Enabled" flag in their settings.
  - b. If BCS queries Ezetap's API using MID and TID, it can retrieve the merchant's current stack context.
  - c. This method is reliable since the data is always available within Ezetap's ecosystem.
2. Razorpay DCS Flag (Single Stack Side)
  - a. On Razorpay's side, a DCS flag is used to indicate whether a merchant is in Single Stack.
  - b. However, relying only on this DCS flag can lead to errors because:
    - i. The merchant ID is required to fetch the DCS flag.
    - ii. Merchant ID may not always be available in BCS, especially in scenarios like:
    - iii. The merchant has moved from POS Stack to Single Stack but hasn't yet processed any transactions on Single Stack.

- iv. The merchant still has unsettled transactions in the POS Stack.
- v. In such cases, BCS won't have the merchant ID, making DCS unreliable.

## 5.5 Updates in respective stack

1. For Single Stack transactions, Gateway Capture and Merchant Capture will continue to function as they do today.
2. For Dual stack transactions, based on the source in the transaction table, Gateway capture and Merchant capture will be triggered from Batch clearing service.
3. For POS and Dual Stack transactions, an additional step is required because transaction updates need to be reflected in the Ezetap database. This api should accept a list of transactions and their update status. It should also accept MID and TID.

## 5.6 New API on Ezetap Side

1. Terminal Lock/Unlock API - To lock or unlock a terminal based on the batch clearing process

Expected Request/Response

Unset

```
{
  "tid": "123456",
  "mid": "MID12345",
  "lock_status": true // true = Lock, false = Unlock
}
```

Unset

```
{
  "success": true,
  "message": "Terminal locked successfully"
}
```

2. Enable Key exchange API - To enable key exchange between in Ezetap for terminals involved in batch clearing.

Expected Request/Response

Unset

```
{
  "tid": "123456",
  "mid": "MID12345",
  "enable_key_exchange": true // true = Enable, false = Disable
}
```

Unset

```
{
  "success": true,
  "message": "Key exchange enabled successfully"
}
```

3. Get Merchant context API - To fetch the merchant's current stack context (Single Stack or POS Stack) in real-time

Expected Request/Response

Unset

```
{
  "mid": "MID12345",
  "tid": "123456"
}
```

Unset

```
{
  "mid": "MID12345",
  "tid": "123456",
  "stack_type": "single_stack" // OR "pos_stack"
}
```

4. Update transactions API - To update transaction statuses Ezetap after batch clearing.

## Expected Request/Response

Unset

```
{
  "merchant_id": "MID12345",
  "terminal_id": "123456",
  "transactions": [
    {
      "transaction_id": "txn_001",
      "status": "captured"
    },
    {
      "transaction_id": "txn_002",
      "status": "failed"
    }
  ]
}
```

Unset

```
{
  "success": true,
  "message": "Transactions updated successfully"
}
```

5. API to fetch terminalId, merchantId from TID and MID - To fetch the razorpay's terminalId and merchantId from TID and MID. If a merchant is moved from pos to single stack and he hasn't done any transaction, batch clearing won't be having terminalId and merchantId. These are required to perform terminal locking, key exchange etc.

## Expected Request/Response

### 5.6 Dual stack omni merchant transactions

Currently Ezetap is handling batch-clearing for omni dual stack merchants. PCP event streaming flow can be used to consume dual stack transactions directly in batch-clearing service instead of consuming from Ezetap Queue. We can add additional checks on Queue that if the transactions are already consumed in batch-clearing, we can skip them.

For dual stack refunds, we will be entirely relied on Ezetap Queue only.

## 5.7 New Setting in Ezetap

To support this migration and rollout plans, we need to add new settings on the Ezetap side. We need two settings, one at merchant level and other at PG level. These settings will be used to decide if batch-clearing is happening at a new service. If this is true, data will be pushed to Queue after payments/refunds and those particular terminals will be skipped during settlement crons.

1. Merchant Level Setting - If enabled, this will push payments and refunds data into Queue and these merchant terminals will be skipped from Settlement cron.
2. PG Level Setting - If enabled, all the transactions of this PG will be pushed to Queue and all the terminals of this PG will be skipped from Settlement cron.

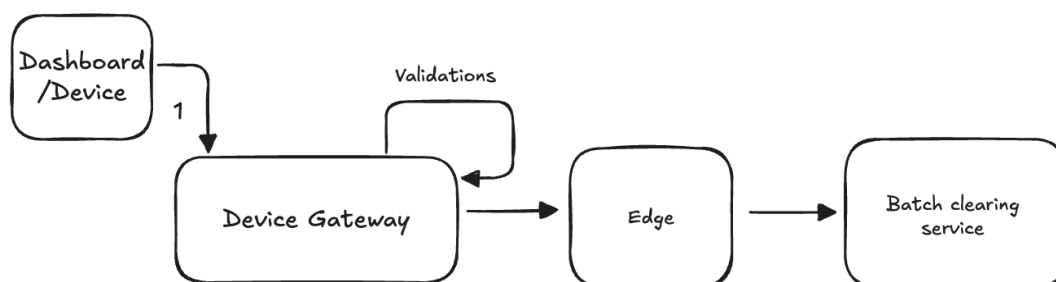
PG setting will act as parent setting for merchant setting. PG setting will help in migrating complete PG to batch-clearing service whereas merchant setting may be for few selected merchants.

## 5.8 Middleware Wrapper Changes

Current Middleware wrapper has to be changed to accommodate settling all stack transactions.

## 5.9 Manual Settlements [Planning for next phase]

1. Currently, manual settlements are performed via the Ezetap dashboard and on the device.
2. We plan to migrate these manual settlement processes to the batch-clearing service. To achieve this, we can build a wrapper on the Ezetap side that forwards manual settlement requests to the batch-clearing service based on configuration. Additionally, the batch-clearing service should expose APIs that allow manual batch closure by using the Terminal ID (TID) and Merchant ID (MID).
3. There are some validation on Ezetap API for Manual settlements like userAuth, userRole and org setting for manual settlement. We need to move these validations on device gateway and after validation, device gateway can call batch clearing service via Edge.
4. Manual settlement merchants are skipped in Cron based settlements, we need to provide provision for this also in batch clearing service.



## 5.10 Merchant/Org Level Settlement Config [Planning for next phase]

On the Ezetap side, there's support for batch-clearing schedule config to be configured at merchant level. Merchant level crons will run before PG level crons. There's also check if the merchant config is skipped or not run, PG cron covers that. The same should be implemented in batch-clearing service.

Total number of HDFC merchants - 3.6 Lakh

Total number of HDFC merchants aggregator - 15k

Current merchants where Merchant/Org level settlements is enabled: 4525

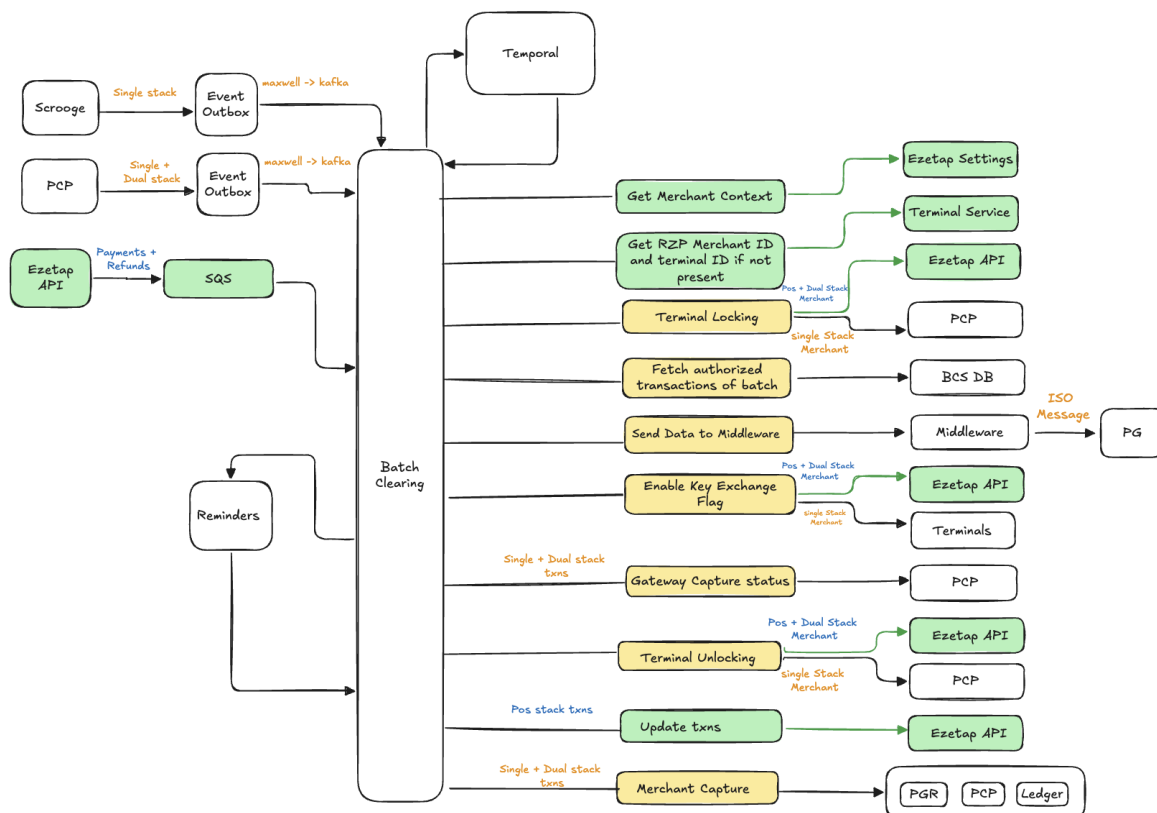
Current HDFC merchants where this setting is enabled: 601

Current HDFC bank acquired merchants where this setting is enabled: 400

## 5.11 Complete Approach

1. Ezetap will push transaction data to the Batch Clearing Service (BCS) via Queue.
2. BCS will consume this data and create batches against MID (Merchant ID) and TID (Terminal ID). Transaction source will also be added in the meta\_data column of the transactions table and source of the merchant will be set in the batch table.
3. Batch schedules will be set as per PG (Payment Gateway) configurations to trigger batch closures at the defined times.
4. When batch closure starts, BCS will fetch the merchant's context from Ezetap settings.
5. Based on which stack the merchant belongs to, different actions will be triggered:
  - a. For Single Stack merchants → Terminal locking & key exchange enablement will be handled via PCP and Terminal service .
  - b. For POS Stack merchants → The same operations will be performed via Ezetap API. A new api needs to be created for terminal locking and enabling key-exchange.
6. Once the batch closure is triggered, batch-clearing will send the complete transaction data to the middleware service.
7. The current transaction processing wrapper in middleware must be updated to accept both single-stack and POS stack transactions for settlement.
8. Once middleware settles the transactions, their status will be updated to "Settled" in the middleware database (mware\_db).
9. Terminal unlocking will be performed on the respective stack where it was locked before.

10. Once the middleware call is successful, Batch Clearing Service (BCS) will group transactions based on their source
- For Single Stack transactions → Standard Gateway Capture & Merchant Capture processes will be followed.
  - For POS Stack transactions → A new API call will be made to Ezetap to:
    - Mark transactions as "Settled" in Ezetap's database.
    - Mark the terminal as "Settled" to ensure smooth reconciliation.
  - For Dual stack transactions → A new API call will be made to Ezetap to:
    - Mark transactions as "Settled" in Ezetap's database.
    - Mark the terminal as "Settled" to ensure smooth reconciliation.
    - Standard Gateway Capture & Merchant Capture processes will be followed.
11. Batch gets successfully closed and transactions will be updated to capture. The same process will repeat whenever a new transaction is made and batch-clear happens.





## 6. Dependencies

If your application needs help from other teams for new APIs, designs, dependent features, changes in platform tools, do call out the dependencies with those teams at an early stage before the development starts to ensure that those wouldn't impact your development timeline.

*Callout upstream and downstream dependent services.*

| #  | Service Name                | Dependency                                           | SLA (If applicable) | POC for service |
|----|-----------------------------|------------------------------------------------------|---------------------|-----------------|
| 1. | Batch clearing service      | Workflow Changes<br>Queue Consumer<br>Other changes  |                     |                 |
| 2. | Ezetap API                  | Queue Producer<br>New APIS                           |                     |                 |
| 3. | Middleware                  | Wrapper Changes                                      |                     |                 |
| 4. | <del>Device Gateway</del>   | <del>Manual Settlement changes</del>                 |                     |                 |
| 5. | <del>Edge</del>             | <del>Manual Settlement changes</del>                 |                     |                 |
| 6. | Terminal Service/Ezetap API | API to fetch terminalId, merchantId from MID and TID |                     |                 |

## 7. Schema Changes

There are no schema changes in batch-clearing service. Some columns are already made optional for Amex batch clearing which will be used in this migration also. But the current meta\_data columns in the **transaction** table and **batch** table will now accommodate new data to support this migration.

1. In the Transaction table, we will be storing transaction sources (single\_stack, dual\_stack or pos\_stack) so that it can be used to update these transactions on their respective stacks.
2. In the Batch table, merchant current stack will be stored (single\_stack, dual\_stack or pos\_stack) so that it can be used to perform terminal locking and enabling key exchange on their respective stacks.

Optional:

1. Create a new source column in the batch-clearing transactions table where we store if this is from single-stack or pos-stack. Currently we can store this in the meta-data column which is json. At any time if we need to query on this source, we can add the filter on this new column.

## 8. Rollout Plan

### 8.1 Dry Run Build

1. A dry run deployment will be carried out on the batch-clearing side, allowing data to be consumed from Queue without actually triggering PG settlement calls.
2. Config on PG and acquirer level to enable dry run.
  - a. Need to store dry\_run flag on PG and acquirer level in payment\_gateway\_config table of batch-clearing service.
    - i. Application level config??
  - b. This flag helps in performing dry runs for a particular payment gateway.
  - c. The current workflow has to be changed to use this flag to perform a dry run, meaning batch-clearing happens without making any downstream calls.
  - d. This config and workflow also helps in performing dry run tests for multiple PG's and not requiring any code changes for future use cases.
3. This phase helps in closely monitoring production data and request patterns while identifying potential issues without causing disruptions.
4. Logs and metrics will be analyzed to verify that all required transactions are processed correctly.
5. Any discrepancies, missing data, or unexpected behavior will be addressed before proceeding with actual merchant onboarding.

### 8.2 Merchant Level Rollout

1. A phased rollout at the merchant level can be implemented, where specific merchants are batch-cleared using the batch-clearing service.
2. To enable this, an identifier must be assigned at the merchant level or terminal level, ensuring that only selected merchants' data is pushed to Queue.
3. The same identifier must be used to prevent Ezetap from picking up these terminals in the settlement cron.
4. Batch clearing will be managed entirely by the batch-clearing service.
5. Gradual ramp-up can be done, starting with a small set of merchants and expanding based on system performance and stability.

### 8.3 PG Level Rollout

1. A complete PG-level rollout can be planned by disabling the Settlement Cron for the targeted PG, ensuring batch clearing does not happen on Ezetap's side.

2. Configuration changes on PG level will ensure that only data for selected PGs is pushed into Queue.
3. Batch clearing for these PGs will be exclusively handled by the batch-clearing service.
4. A phased approach will be followed where one **PG** is onboarded at a time, with necessary validations before moving to the next.

## 9. Rollback Strategy

In case we need to stop the migration and revert to the previous system, we need a structured rollback approach to ensure minimal disruptions in transaction settlements. The rollback strategy is divided into two key areas: Ezetap-side rollback and Batch-Clearing Service rollback.

1. Rolling Back from Ezetap Side
  - a. If we decide to stop the migration from Ezetap, we need to disable the new settings that push transaction data to Queue.
  - b. Once the settings are disabled:
    - i. Ezetap will stop pushing transactions to batch-clearing service.
    - ii. Transactions will continue being picked up during the settlement cron, as per the old workflow.
    - iii. This ensures that no further transactions are diverted to batch-clearing, effectively restoring the previous settlement mechanism.
2. Rolling Back from Batch-Clearing Service Side
  - a. If the migration is stopped at the batch-clearing service level, we need to handle transactions already consumed into the system.
  - b. For Merchant-Level Rollback:
    - i. Batches for individual merchants can be manually disabled without closing the batch.
    - ii. This prevents these transactions from being processed further, allowing the merchant's settlement to happen via the original flow.
  - c. For PG-Level Rollback:
    - i. If we need to roll back at the payment gateway (PG) level, we must disable the configuration in the payment\_gateway\_config table within the batch-clearing service.
    - ii. This ensures that no transactions are picked up for batch processing under this PG.
    - iii. Any pending transactions in batch-clearing can be manually reviewed or settled before fully switching back.
3. Handling Data Fixes for Polluted Data
  - a. If batch-clearing encounters any unexpected issues (e.g., incorrect batch closures, missing transactions, or duplicate settlements), data fixes might be required to correct polluted data.
  - b. This may involve:

- i. Manually adjusting database records to ensure consistency.
- ii. Reverting incorrectly settled transactions to their previous state.
- iii. Resyncing affected transactions with Ezetap to avoid settlement mismatches.

## 10. Traffic Estimates

1. Which PG's are we targeting for go live - HDFC
2. Are we targeting all the merchants - Currently the plan is to migrate HDFC Bank acquired merchants
3. Total number of HDFC merchants - 3.6 Lakh
4. Total number of HDFC merchants aggregator - 15k
5. Total number of HDFC terminals - 302884
6. Total number of HDFC Agg terminals - 14k
7. Total number of payments per day -
8. Total number of refunds per day -

## 11. Batch clearing service stability

### 1. Improving Health Checks

Right now, the health checks only verify the database connection and maximum thread count. However, the service depends on Temporal Workers and Kafka Consumers, which are not being monitored. If they stop working, it may take hours to detect the issue. Currently, checking their status requires manual intervention.

- a. Introduce deeper health checks that monitor the state of Kafka Consumers and Temporal Workers.
- b. Set up automated alerts to notify if workers stop processing tasks. Configure Kubernetes Liveness Probes to check these components every minute. If they fail, Kubernetes should automatically restart the affected pods.

### 2. Moving to a Dedicated Database

The Batch Clearing Service is using a shared database cluster (bvs), which can lead to performance issues due to unpredictable query loads from other services. There is also limited visibility into how the database is performing specifically for batch clearing operations.

- a. Migrate to a separate database instance dedicated to batch clearing.
- b. Implement database monitoring tools to track query performance and optimize slow queries.
- c. Set up read replicas to distribute the database load.

### 3. Load Testing and Performance Benchmarking

There is no clear understanding of how much traffic the service can handle before it reaches its limit. This makes it difficult to scale efficiently.

- a. Conduct load testing.
- b. Identify the maximum number of transactions the system can process per second.
- c. Fine-tune pod counts, Kafka partitions, and Temporal worker instances based on test results.

#### 4. Implementing Autoscaling

At the moment, there is no auto-scaling configured for the service. If traffic spikes, the system may slow down or crash due to resource exhaustion.

- a. Enable Horizontal Pod Autoscaling (HPA) based on CPU utilization, Kafka message backlog, and Temporal Workflow queue size.
- b. Set up Cluster Autoscaler to provision additional nodes when needed.
- c. Define clear scaling thresholds to prevent excessive pod creation while ensuring stability.

#### 5. Replacing API-Based Reminders with Temporal Schedules

Batch clearing is currently triggered using reminders via API calls. Every time batch clearing is needed, an API call is made to register the reminder, and another is required to trigger the process when the time comes. This adds unnecessary dependencies and potential delays.

- a. Replace the existing system with Temporal Schedules to handle batch clearing events natively within Temporal.
- b. Reduce API dependencies by letting Temporal handle scheduled tasks directly.
- c. Implement retry logic at the workflow level rather than relying on external retries.

#### 6. Adding Tracing and Observability

The service relies heavily on logs for debugging, which makes it difficult to trace issues across multiple systems like Kafka, Temporal, and the database. Without proper tracing, diagnosing failures can take hours.

- a. Integrate distributed tracing.
- b. Track request flows across different components, including Kafka, Temporal, and API calls.
- c. Implement Application Performance Monitoring (APM) to track real-time system health.

#### 7. Improving Fault Tolerance

The service does not have proper circuit breakers or rate limits in place. If a downstream service (like a middleware) becomes unresponsive, the system keeps retrying requests, leading to unnecessary load and potential failures.

- a. Introduce Circuit Breakers to stop making requests when a service is consistently failing.
- b. Implement Rate Limiting to control how frequently external APIs are called.
- c. Use Bulkheads to isolate different parts of the system to prevent cascading failures.

## 12. Open questions

1. SQS for both Ezetap and Razorpay service - Compatibility of SQS with Razorpay and Ezetap needs to be checked if there are no access issues.
  - a. SQS cross account access can be enabled. Need to check with devops will they allow this
  - b. Cross account access is possible but needs CAB approval as confirmed by Devops - [Thread Link](#)
2. Rollback: Add rollback steps - **DONE**
3. Merchant context propagation during migration - event based - **DONE**
4. Proper dry run config at PG level or merchant level. - **DONE**
5. In case of PG level rollout, should we maintain a config on PG level in Ezetap. Like only these PG transactions will be pushed to Queue. Or Should we maintain this at Batch-clearing like only these PG transactions will be consumed.
  - a. Better to maintain at Ezetap level, else every pg data will flow through Queue
6. Ezetap performs some actions before sending data to middleware like reversing transactions, validations on org level and acquirer level etc. Should batch-clearing do these changes? Or should we eliminate them if not required.
  - a. Reversing transactions should be ignored as they are currently also not happening in Amex batch clearing and single stack batch clearing. Separate reversal crons run in Ezetap API to ensure this reversal.
  - b. Zero transactions settlements - Currently for any unsettled terminals, even if there are 0 authorized transactions, we are marking the terminal as settled and enabling key exchange. This provision has to be provided in batch-clearing service also.
7. Proper migration SOP and Experience
8. Comparison between pre and post migration
9. Rollback

## 13. Migration Experience

1. Code Deployment (Ezetap Service)
  - a. As part of the regular release cycle, new logic is introduced:
  - b. A new queue to push payment data.
  - c. Supporting APIs for batch clearing service.
  - d. Cron skipping logic to prevent duplication in batch processing.

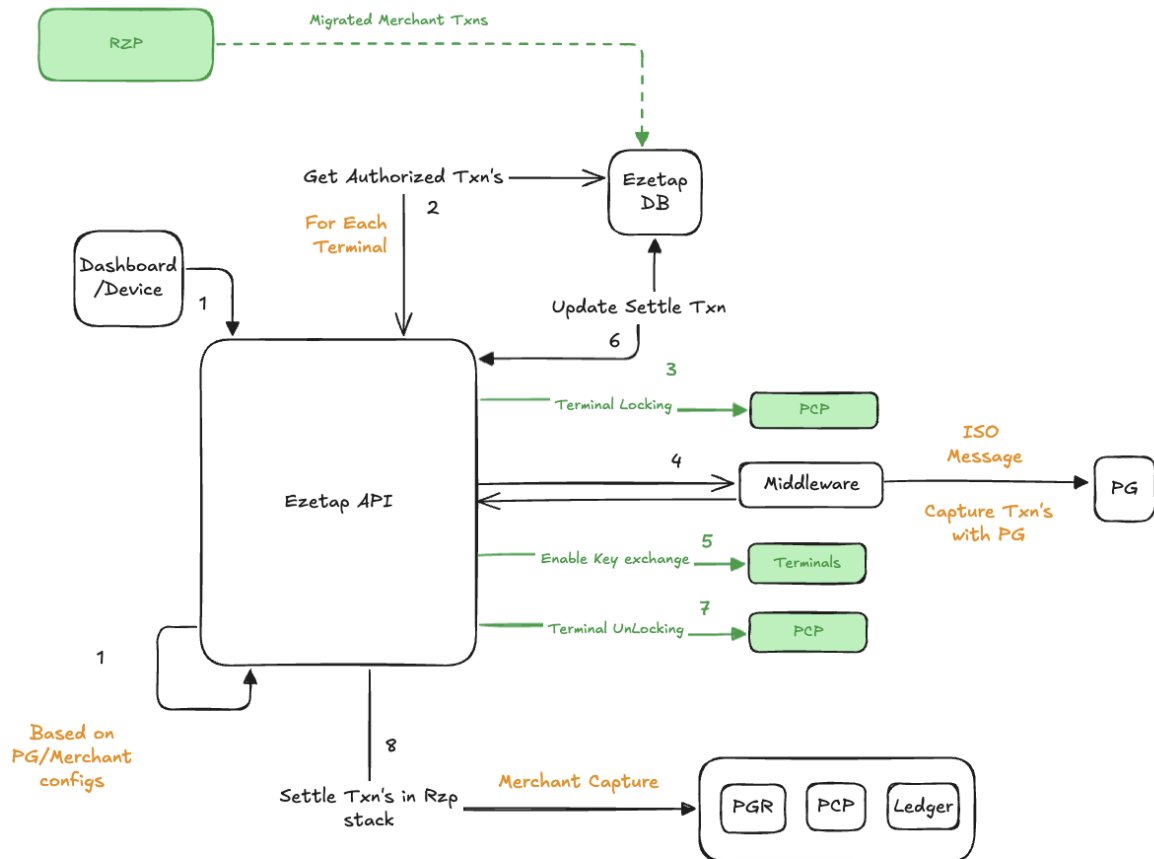
2. Code Deployment (Batch clearing Service)
  - a. Batch clearing changes will be deployed to consume the transactions and create batch accordingly
  - b. This can go live independently
3. Config-Based Rollout
  - a. Once deployment is successful and no issues are observed, we gradually enable setting on a terminal level manually for a subset of merchants after their EOD batch clearing is completed on the Ezetap stack.
  - b. This ensures that new transactions for these merchants are routed through the new queue, to be picked up by the batch clearing service. The same setting will be used to skip these terminals on Ezetap cron
  - c. Batch clearing service will consume these transactions and batches will be created thereby scheduling reminders.
  - d. Once reminders are received, the batch clearing service will trigger the batch clearing process.

## 14. Migrating Batch clearing responsibilities post merchant migration

Since dual writes are planned for merchant migration, We can think of a possibility to migrate this batch clearing responsibility after merchant migration is complete. Ezetap will now have both pos stack and new stack transactions. This can help in batch clearing migrated merchants also via Ezetap system but with some changes needed. The process looks like

1. Ezetap will continue to do batch clearing for pos stack and migrated stack also. As Ezetap will now have migrated stack transactions with dual write, it can close batch clearing for migrated merchants.
2. Legacy settlement cron in Ezetap should be modified to handle terminal locking and key exchange in the new stack. This will require merchant level identification and based on that, new api's will be called.
3. This will still help merchant migration to happen flexibly without thinking much on closing batch-clearing and then migrating merchants.
4. Batch clearing service will not consume migrated merchant transactions.
5. Once the stable state of merchant migration is achieved, a cutover will be done from Ezetap to batch clearing service.
6. New configs to be added so that batch clearing can consume this migrated merchant transactions and process batch clearing. Ezetap crons should be skipped if this config is enabled.
7. The rollout plan will remain similar to above. We will first push Ezetap side changes in settlement cron and enable migrated merchant settings. Once we decide to cutover to batch clearing service, a new setting will be added to merchant level in Ezetap to skip it from settlement cron. This setting should be added just after batch clearing is completed and there are no pending transactions in Ezetap and all the new transactions can be processed in batch clearing service.

8. Rollback plan will remain similar to above rollback plan where we disable the config and Ezetap starts picking up these batch clearing. Already consumed transactions in batch clearing service should be manually disabled.



## Rollout Plans



Phase 1: To help merchant migration

- a. New settings in Ezetap to identity migrated merchants
- a. Changes in Ezetap to settle both pos and migrated merchant transactions
- b. New config in batch clearing service to not consume these migrated merchant transactions

Phase 2: To migrate batch clearing responsibilities

- a. New settings in Ezetap to stop settlements. This setting should be added just after batch clearing is completed and there are no pending transactions in Ezetap and all the new transactions can be processed in batch clearing service.
- b. Config change in batch clearing service to settle transactions.

## **Rollback Plans**

Rollback plan will remain pre-migration to above rollback plan where we disable the config and Ezetap starts picking up these batch clearing. Already consumed transactions in batch clearing service should be manually disabled.

## **Solve for voids and refunds for earlier transactions**

1. Once the responsibility is migrated to batch clearing service, new transactions can be consumed and processed in batch clearing service.
2. Voids cannot happen since we will be migrating to batch clearing after capture.
3. For refunds, we need to add a callback from Ezetap to batch clearing, since this data will be missing in batch clearing. Currently we don't support refunds for HDFC, we can decide its priority.

## **15. Pre and post migration - Comparison**

### **Pre-Merchant Migration**

1. Pros
  - a. Flexible Merchant Migration Timing: Enables migrating merchants at any time during the day, without waiting for End-of-Day (EOD) batch closure on the existing stack.
  - b. Platform-wide Coverage: Batch clearing can process transactions from both stacks, making it a platform-level service instead of stack-specific.
  - c. Smooth Dependency Shift: Helps in moving batch clearing dependency from Ezetap stack to Razorpay stack early in the migration process.
2. Cons
  - a. Requires additional effort to build all-stack batch clearing logic.

- b. Slightly increases complexity, as transactions must be handled across both stacks during the transition.
- 3. Effort Estimation - On high level, this will have a bit longer effort since a new queue addition, workflow changes, new apis etc.

## **Post-Merchant Migration**

- 1. Pros
  - a. Simplified Development: Avoids building batch clearing logic for both stacks. Merchants are onboarded directly to the Razorpay stack, and existing single-stack batch clearing can be reused.
  - b. Less Immediate Coordination: Since all merchants are on one stack, there's no need for cross-stack clearing logic.
- 2. Cons
  - a. Should be relied on legacy Ezetap infrastructure - Changes to be made on Ezetap side
  - b. Might impact merchant migration end goal
  - c. Need to handle refunds for earlier payments - After migration
  - d. Highly Dependent on data replication
- 3. Effort Estimation - This will have relatively low efforts

## **16. Future Scope**

### **Org Level Settlements**

Currently only 600 HDFC merchants have this setting enabled. To migrate this logic also, we need to build this feature in a batch-clearing service (developed partially and not tested) and we need to migrate these settings to batch clearing systems.

### **Manual Settlements**

Currently HDFC merchants are not given Manual settlement feature. Verified this in the settings table with HDFC org code. If we need to migrate this Manual Settlements, we need to migrate these settings to batch clearing systems (New or existing tables to store these config) and then build Device gateway -> Edge -> Batch clearing route.

Cron portal migration \*\*